

VITyarthi - Build Your Own Project

School Management System

A Python and SQLite based student record manager

PROJECT REPORT

Student name	Amey Narendra Sure
Registration number	26MEI10013
Course / Subject	Python Essentials / CSE1021
Faculty	Ajay Kumar Phulre
Institution	VIT (VITyarthi platform)
Date	30 September 2026
GitHub repository	https://github.com/Amey-Narendra-Sure-16/School-Management-System

Table of Contents

1. Cover Page
2. Introduction
3. Problem Statement
4. Functional Requirements
5. Non-functional Requirements
6. System Architecture
7. Design Diagrams (Use Case, Workflow, Sequence, Class/Component, ER)
8. Design Decisions & Rationale
9. Implementation Details
10. Screenshots / Results
11. Testing Approach
12. Challenges Faced
13. Learnings & Key Takeaways
14. Future Enhancements
15. References

2. Introduction

Schools keep a large amount of routine data - above all, records of the students they teach. In many small institutions this is still stored in paper registers or scattered spreadsheets. The result is slow look-ups, duplicate entries, and no control over who may view or change the data.

The **School Management System** is a compact command-line application that addresses this problem. It is written in Python and stores its data in an SQLite database, so it needs no server and no third-party packages. After signing in, an authorised user can add, search, list, update and delete student records through a simple numbered menu. All input is validated, and all database access uses parameterised SQL.

The project applies core concepts of the course: functions and modular decomposition, dictionaries and dispatch tables, string and integer handling, control flow, exception-safe input loops, and relational database access from Python (CRUD).

3. Problem Statement

Manual or ad-hoc record keeping in a school leads to (a) slow retrieval of a student's details, (b) duplicate or conflicting roll numbers, (c) difficulty correcting or removing records, and (d) no restriction on who can access the data. A lightweight, offline, login-protected tool is needed that lets staff create and maintain student records reliably.

Objectives

- Provide a login gate so that only known accounts can use the system.
- Support the full CRUD life cycle for student records.
- Prevent bad data: blank fields, non-numeric IDs and duplicate roll numbers.
- Persist data between sessions using SQLite.
- Keep the code small, readable and easy to extend.
- Document the design with standard UML and architecture diagrams.

Scope and target users

In scope: login, add, search, list, update and delete of student records via a console interface. **Out of scope:** attendance, marks, fees, GUI/web front-end and network access. **Users:** Principal, Teacher/office staff, and a Student/guest account.

4. Functional Requirements

The system provides five data modules plus authentication (six major functional modules), exceeding the minimum of three required.

ID	Module	Requirement	Input	Output
FR1	Authentication	Verify username and password against the stored accounts; allow at most 3 attempts.	Username, password	'login successful' or remaining tries; exit after 3 failures
FR2	Add student	Create a record; reject an existing roll number.	Roll no., name, class, section, guardian phone	'saved' or 'that roll number is already taken'

ID	Module	Requirement	Input	Output
FR3	Search student	Retrieve a single record by ID.	Student ID	Record row or 'no student with that id'
FR4	Show all	List all records ordered by ID with a count.	-	'total records: N' and rows
FR5	Update student	Change one of name, class, section or phone.	ID, field choice 1-4, new value	'updated', 'record not found' or 'not a valid option'
FR6	Delete student	Remove a record by ID.	Student ID	'deleted' or 'no such student_id'
FR7	Input handling	Re-prompt on blank text or non-numeric numbers.	Any prompt	Error message and re-prompt

User workflow

Start program → log in (max 3 tries) → menu is displayed → user chooses an option 1-5 → the system validates input, runs the database operation and prints the result → returns to the menu until the user chooses 6 (exit), when the database connection is closed.

5. Non-functional Requirements

Category	Requirement	How it is met
Security	Only authenticated users may access data; queries must be safe from injection.	Login with 3-attempt limit; all SQL uses '?' placeholders (the update statement whitelists column names from a fixed list).
Reliability / Data integrity	No duplicate IDs; changes must not be half-written.	PRIMARY KEY on student_id, duplicate check via find(), and 'with con:' transactions that commit or roll back.
Usability	Clear prompts and messages; easy to recover from mistakes.	Numbered menu, plain-language error messages, re-prompt loops instead of crashes.
Error-handling strategy	Bad input must never crash the program.	get_text() and get_number() loop until valid; invalid menu choices and missing records are reported.
Performance	Operations should feel instant for school-sized data.	Single-row look-ups use the primary-key index; SQLite is embedded so there is no network latency.
Maintainability	Code easy to read and extend.	Small single-purpose functions, dispatch dictionary for the menu, helper functions find() and show().
Resource efficiency / Portability	Run on any basic machine without installation.	Standard library only; one .db file; runs on Python 3.8+ on Windows, Linux or macOS.

6. System Architecture

The application is a single-process, three-layer design. The **presentation layer** is the console (banner, menu, prompts). The **logic layer** contains the authentication function, the menu dispatcher, input validators and the five student operations. The **data layer** consists of two small helper functions and the sqlite3 connection to `school_data.db`.

System Architecture (3-layer, single process)

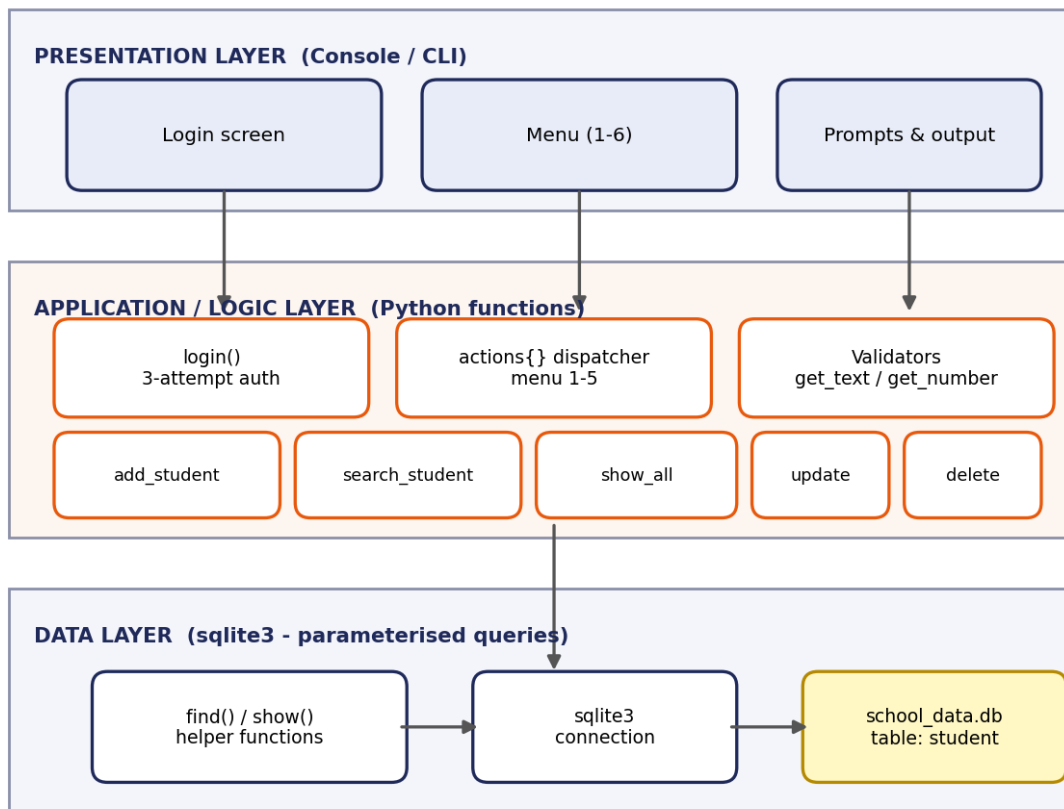


Figure 1 - System architecture

7. Design Diagrams

7.1 Use Case Diagram

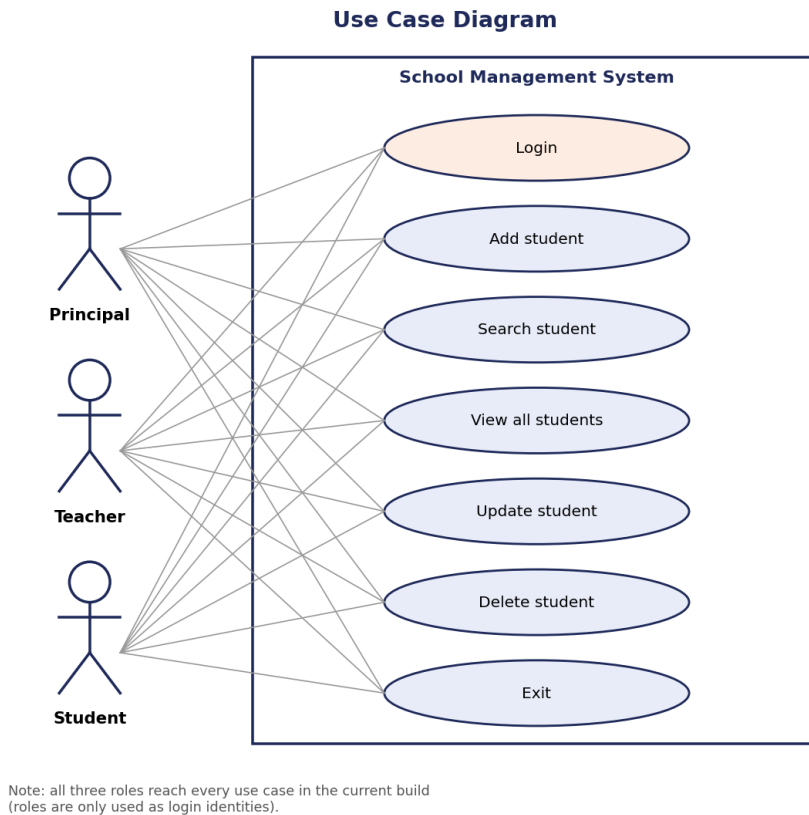


Figure 2 - Use case diagram

Three actors sign in through the same login use case. In the current version every role can reach every operation; roles differ only by credentials.

7.2 Workflow Diagram

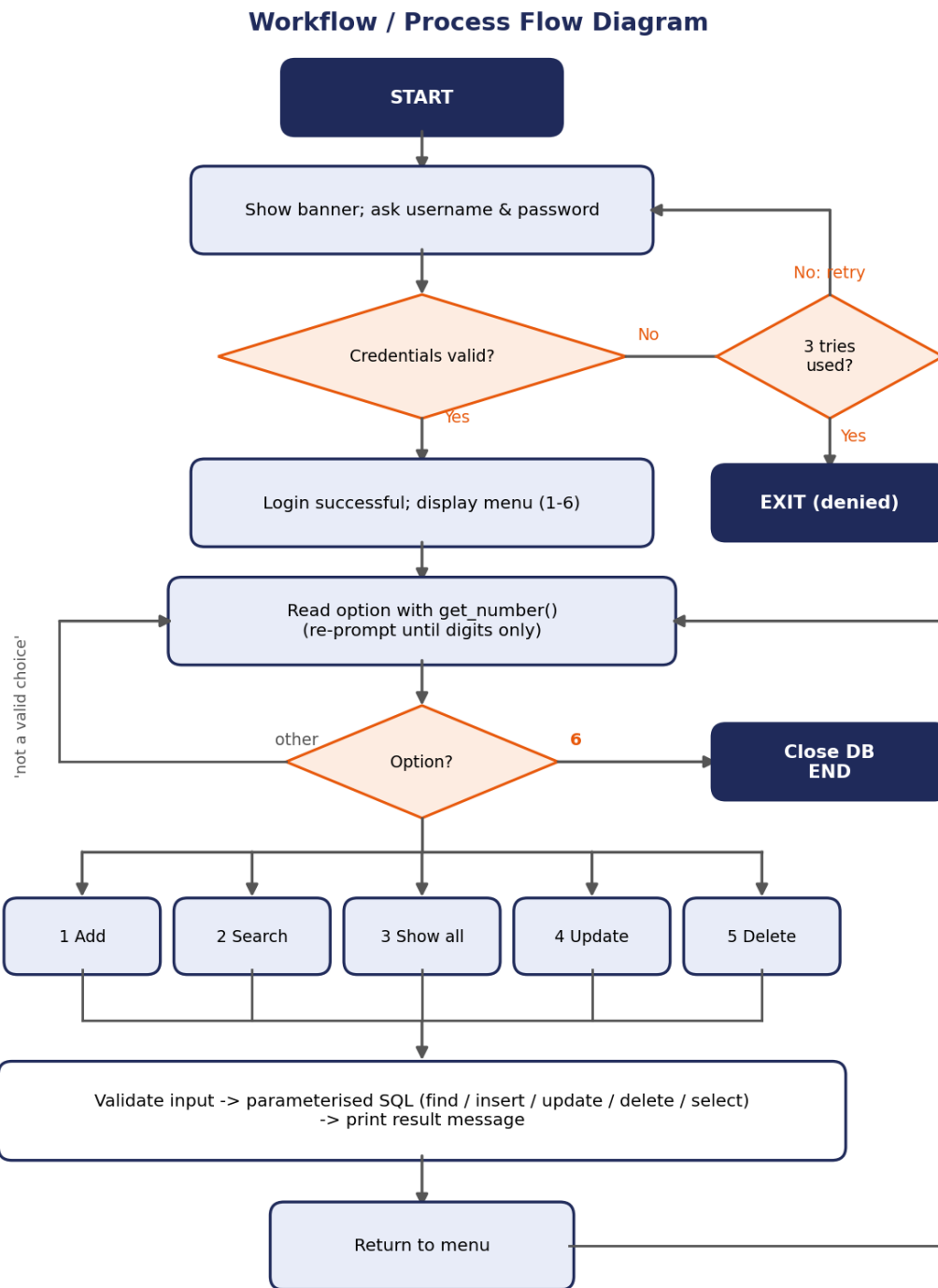


Figure 3 - Process flow of the application

7.3 Sequence Diagram

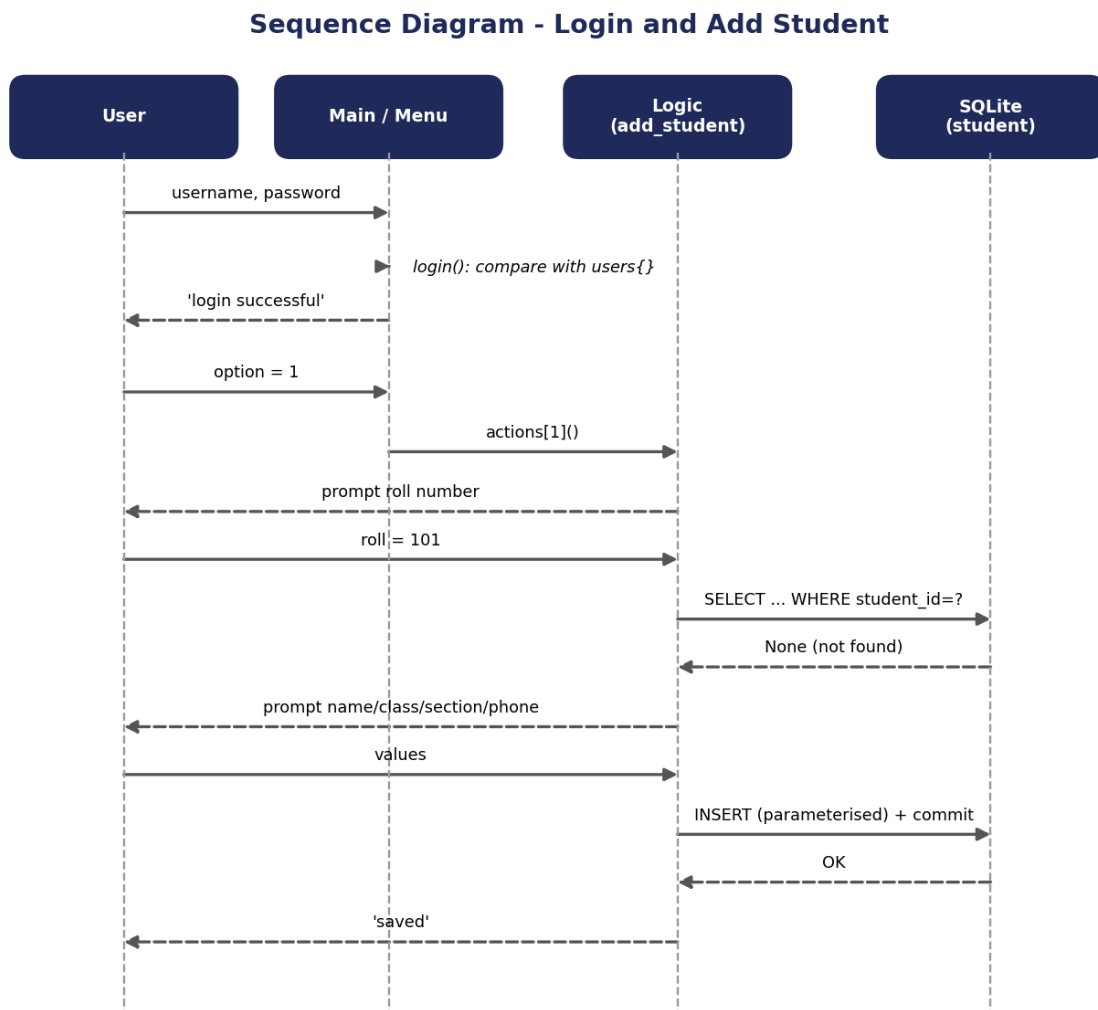


Figure 4 - Sequence: login followed by adding a student

7.4 Class / Component Diagram

The program is written in a procedural style, so a component diagram is used instead of a class diagram. Each box is a group of related functions.

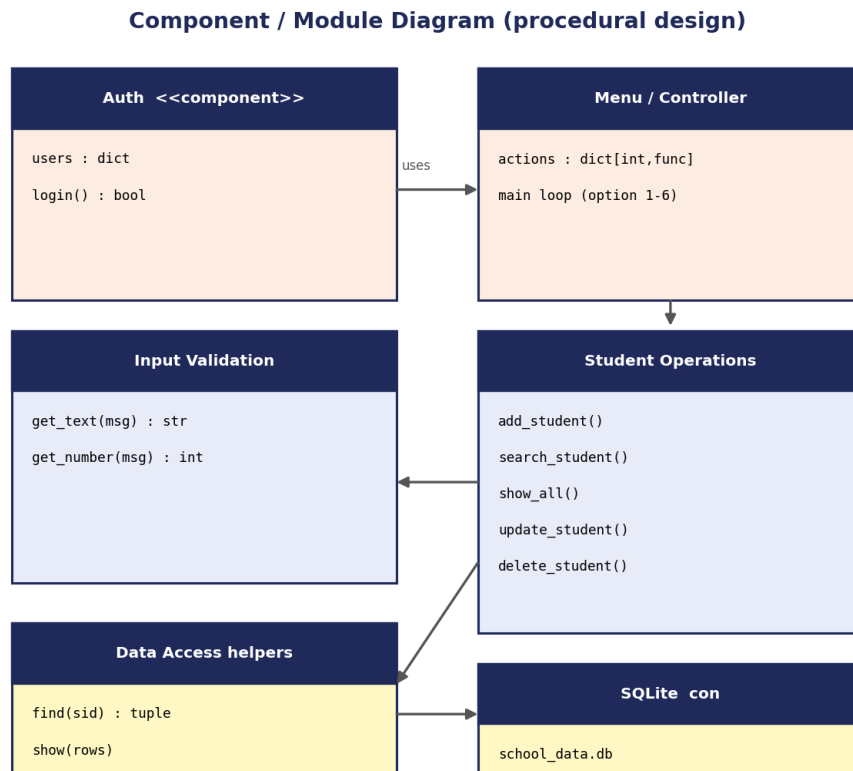


Figure 5 - Component / module diagram

7.5 ER Diagram and Schema

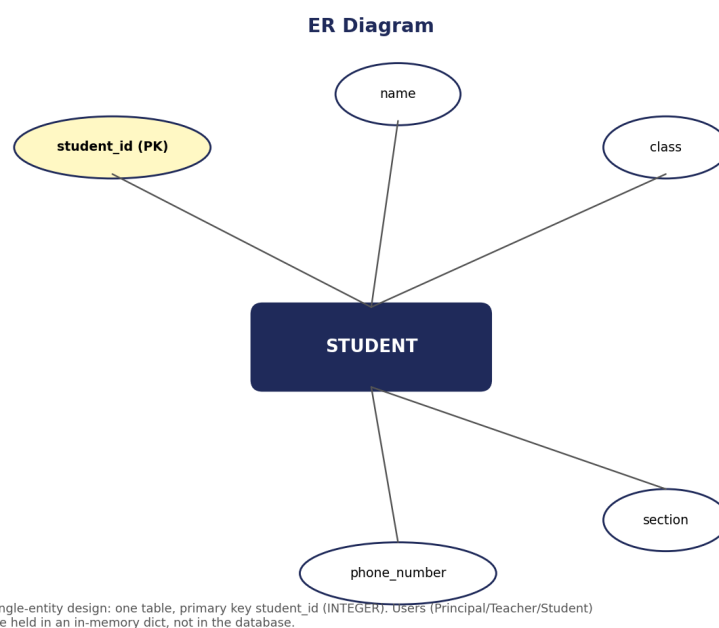


Figure 6 - ER diagram

Column	Type	Constraint	Description
student_id	INT	PRIMARY KEY	Roll number / unique ID
name	VARCHAR(30)	-	Student's full name
class	VARCHAR(5)	-	Class / grade
section	VARCHAR(5)	-	Section
phone_number	VARCHAR(15)	-	Guardian phone number

```
create table if not exists student (student_id int primary key,
    name varchar(30), class varchar(5), section varchar(5), phone_number varchar(15))
```

8. Design Decisions & Rationale

Decision	Rationale
SQLite instead of text/CSV files	Provides real CRUD, a primary key, transactions and persistence with zero setup; part of Python's standard library.
Dispatch dictionary actions = {1: add_student, ...}	Replaces a long if/elif chain; adding a feature means writing one function and one dictionary entry.
Parameterised queries everywhere	Prevents SQL injection and handles quotes in names safely.
Column name chosen from a fixed list in update	Column names cannot be parameterised in SQL, so the f-string is fed only from a whitelist, never from user text.
Reusable validators get_text / get_number	One place enforces 'not blank' and 'digits only', keeping each feature function short.
'with con:' around writes	Commits on success and rolls back on error, protecting data integrity.
Console interface	Keeps focus on logic and data handling; runs anywhere and is easy to test with piped input.
Duplicate check with find() before insert	Gives a friendly message instead of a raw IntegrityError.

9. Implementation Details

The source (`school_management.py`, about 60 lines) is organised as follows:

Element	Details
Setup	Imports <code>sqlite3</code> ; defines the users dictionary (Principal, Teacher, Student); opens <code>school_data.db</code> and creates the student table if absent.
Validators	<code>get_text</code> loops with the walrus operator until non-blank; <code>get_number</code> loops until <code>str.isdigit()</code> , then returns an int.
Data helpers	<code>find(sid)</code> returns one row or None; <code>show(rows)</code> prints fields separated by ' '.
CRUD functions	<code>add_student</code> , <code>search_student</code> , <code>show_all</code> , <code>update_student</code> , <code>delete_student</code> - each validates input, runs one SQL statement and prints a result message.
login()	Up to three attempts; returns True on the first match of username and password in <code>users</code> .
Main loop	Prints the menu, reads the option, looks it up in <code>actions</code> , and repeats until option 6; then closes the connection.

Key code excerpt - update with a whitelisted column

```
def update_student():
    sid = get_number('enter student_id to update : ')
    if not find(sid): return print('record not found')
    choice = get_number('change what? 1.name 2.class 3.section 4.phone : ')
    if not 1 <= choice <= 4: return print('not a valid option')
    column = ['name', 'class', 'section', 'phone_number'][choice - 1]
    with con: con.execute(f'update student set {column}=? where student_id=?',
                          (get_text('new value : '), sid))
    print('updated')
```

Running the project

- Requires Python 3.8 or newer; no extra packages.
- Run `python school_management.py`; the database file is created automatically.
- Default logins: Principal / Boss@123, Teacher / Staff@123, Student / guest2026.

10. Screenshots / Results

The output below was captured by running the program with scripted input on a fresh database (prompts appear on one line because input was piped rather than typed). Replace or supplement these with your own terminal screenshots if your evaluator requires images.

Run 1 - successful login and full CRUD session

```
#### SCHOOL MANAGEMENT SYSTEM ####
username : password : login successful

1. add 2. search 3. show all 4. update 5. delete 6. exit
pick an option :
> enter student roll number : name : class : section : guardian phone : saved

1. add 2. search 3. show all 4. update 5. delete 6. exit
pick an option :
> enter student roll number : name : class : section : guardian phone : saved

1. add 2. search 3. show all 4. update 5. delete 6. exit
pick an option :
> enter student roll number : that roll number is already taken

1. add 2. search 3. show all 4. update 5. delete 6. exit
pick an option :
> enter student_id : 101 | Aisha Khan | 10 | A | 9876543210

1. add 2. search 3. show all 4. update 5. delete 6. exit
pick an option :
> enter student_id : no student with that id

1. add 2. search 3. show all 4. update 5. delete 6. exit
pick an option :
> total records: 2
101 | Aisha Khan | 10 | A | 9876543210
102 | Rohan Mehta | 9 | B | 9123456780

1. add 2. search 3. show all 4. update 5. delete 6. exit
pick an option :
> enter student_id to update : change what? 1.name 2.class 3.section 4.phone : new value : updated

1. add 2. search 3. show all 4. update 5. delete 6. exit
pick an option :
> enter student_id : 102 | Rohan Mehta | 9 | Rohan M. | 9123456780

1. add 2. search 3. show all 4. update 5. delete 6. exit
pick an option :
> enter student id to delete : deleted

1. add 2. search 3. show all 4. update 5. delete 6. exit
pick an option :
> total records: 1
102 | Rohan Mehta | 9 | Rohan M. | 9123456780

1. add 2. search 3. show all 4. update 5. delete 6. exit
pick an option :
> not a valid choice

1. add 2. search 3. show all 4. update 5. delete 6. exit
pick an option :
> numbers only please
pick an option :
>
```

Run 2 - login failures then success

```
#### SCHOOL MANAGEMENT SYSTEM ####
username : password : wrong username/password, 2 tries left
username : password : wrong username/password, 1 tries left
username : password : login successful
```

1. add 2. search 3. show all 4. update 5. delete 6. exit
pick an option :

Run 3 - three failed attempts

```
#### SCHOOL MANAGEMENT SYSTEM ####
username : password : wrong username/password, 2 tries left
username : password : wrong username/password, 1 tries left
username : password : wrong username/password, 0 tries left
(program exits without showing the menu)
```

11. Testing Approach

Validation testing was done black-box: each feature and each error path was exercised by running the program with prepared input (typed or piped through stdin) and comparing the output with the expected message. Results below come from the runs shown in section 10.

#	Test case	Input	Expected	Result
T1	Valid login	Principal / Boss@123	login successful	Pass
T2	Wrong password x3	wrong credentials	Tries left 2, 1, 0; exit	Pass
T3	Add student	101, Aisha Khan, 10, A, phone	saved	Pass
T4	Duplicate roll number	101 again	that roll number is already taken	Pass
T5	Search existing	101	Record displayed	Pass
T6	Search missing	999	no student with that id	Pass
T7	Show all	option 3	total records: 2 + rows	Pass
T8	Update field	102, option 3, new value	updated; change visible on search	Pass
T9	Delete	101	deleted; total records drops to 1	Pass
T10	Invalid menu choice	7	not a valid choice	Pass
T11	Non-numeric input	abc at numeric prompt	numbers only please; re-prompt	Pass
T12	Invalid update option / missing ID	option 9; unknown ID	not a valid option / record not found	Not run in captured session - recommended to run manually

Note: the project has no automated unit-test suite yet; adding pytest tests for find(), the validators and the CRUD functions is listed under Future Enhancements.

12. Challenges Faced

- **Safe dynamic update:** SQL cannot parameterise a column name, so a whitelist list indexed by the menu choice was used to avoid injection.
- **Robust input:** making every prompt tolerant of blank and non-numeric entries required reusable validator loops.
- **Duplicate keys:** letting SQLite raise an IntegrityError gave an unfriendly crash, so an explicit find() check was added first.
- **Keeping the code compact yet readable:** balancing short functions and a dispatch table against clarity for a reader new to the code.

- **Access control:** deciding how much role-based logic was practical for a first version led to the limitation noted in section 14.

13. Learnings & Key Takeaways

- Using Python's sqlite3 module for persistent CRUD with parameterised queries and transactions.
- Designing small, single-purpose functions and dispatch tables for clean control flow.
- The importance of input validation and error messages for usability and reliability.
- Translating a working program into design artefacts: architecture, use case, workflow, sequence, component and ER diagrams.
- Reading a program critically to identify security and scalability limits (plain-text credentials, no logging).

14. Future Enhancements

Enhancement	Benefit
Split the code into modules/packages (auth, db, validators, students, main) and add unit tests	Meets the 5-10 module expectation, improves maintainability and testability
Role-based permissions (e.g. Student read-only, Teacher no delete)	Makes the three roles meaningful
Hash passwords (e.g. hashlib/bcrypt) and store users in the database	Removes plain-text credentials from source code
Logging of logins and record changes (Python logging module)	Audit trail and monitoring
Validation of phone number format and class/section values	Higher data quality
Search by name/class, export to CSV, attendance and marks modules	More useful reporting and analytics
GUI or web front-end (Tkinter / Flask)	Better usability for non-technical staff

15. References

- Python Software Foundation. *sqlite3 - DB-API 2.0 interface for SQLite databases*. docs.python.org/3/library/sqlite3.html
- SQLite Documentation. *sqlite.org/docs.html*
- Python Software Foundation. *The Python Tutorial*. docs.python.org/3/tutorial
- OWASP. *SQL Injection Prevention Cheat Sheet*. cheatsheetseries.owasp.org